

Refactoring

Ejercicio 10

Malos olores y refactorings aplicados:

- I. Public Field: En la clase **Cliente** la variable “llamadas” es pública.
- II. Aplico Encapsulate Field.
 - A. Convierto la variable “llamadas” de la clase cliente a privada.
 - B. Creo un método en la clase **Cliente** llamado “getLlamadas” que devuelva las llamadas de la clase **Cliente**.
 - C. Reemplazo las llamadas de la variable “llamadas” por el método “getLlamadas”.
 - D. Creo un método en la clase **Cliente** llamado “agregarLlamada” que reciba una llamada y la agregue a la lista de llamadas.
 - E. Reemplazo las llamadas de “llamadas.add” por “agregarLlamada”.

```

6 public class Cliente {
7     private List<Llamada> llamadas = new ArrayList<Llamada>();
8     private String tipo;
9     private String nombre;
10    private String numeroTelefono;
11    private String cuit;
12    private String dni;
13
14    public String getTipo() {
15        return tipo;
16    }
17    public void setTipo(String tipo) {
18        this.tipo = tipo;
19    }
20    public String getNombre() {
21        return nombre;
22    }
23    public void setNombre(String nombre) {
24        this.nombre = nombre;
25    }
26    public String getNumeroTelefono() {
27        return numeroTelefono;
28    }
29    public void setNumeroTelefono(String numeroTelefono) {
30        this.numeroTelefono = numeroTelefono;
31    }
32    public String getCuit() {
33        return cuit;
34    }
35    public void setCuit(String cuit) {
36        this.cuit = cuit;
37    }
38    public String getDNI() {
39        return dni;
40    }
41    public void setDNI(String dni) {
42        this.dni = dni;
43    }
44
45    public List<Llamada> getLlamadas()
46    {
47        return this.llamadas;
48    }
49
50    public void agregarLlamada(Llamada llamada)
51    {
52        this.llamadas.add(llamada);
53    }
54 }

```

III.

- I. Uncommunicative Name: En el método “registrarUsuario” de la clase **Empresa**, la variable “var” no se explica por sí misma.
- II. Aplico Rename Variable

- A. La renombro como “cliente” y actualizo sus llamadas dentro del método “registrarUsuario”.

```
31 public Cliente registrarUsuario(String data, String nombre, String tipo) {
32     Cliente cliente = new Cliente();
33     if (tipo.equals("fisica")) {
34         cliente.setNombre(nombre);
35         String tel = this.obtenerNumeroLibre();
36         cliente.setTipo(tipo);
37         cliente.setNumeroTelefono(tel);
38         cliente.setDNI(data);
39     }
40     else if (tipo.equals("juridica")) {
41         String tel = this.obtenerNumeroLibre();
42         cliente.setNombre(nombre);
43         cliente.setTipo(tipo);
44         cliente.setNumeroTelefono(tel);
45         cliente.setCuit(data);
46     }
47     clientes.add(cliente);
48     return cliente;
49 }
```

III.

- I. Duplicate Code: En la clase **Empresa** de las líneas 34 a 37 y 41 a 44, existe código muy similar.
- II. Defino un constructor en la clase **Cliente** para evitar código repetido.
 - A. Defino un constructor en la clase **Cliente** con las variables “nombre”, “tipo” y “telefono”
 - B. En el método “registrarUsuario” de la clase **Empresa** defino el constructor de cliente con el nombre, tipo y teléfono.
 - C. Elimino los setters de nombre, tipo y teléfono junto con la variable “tel”.

```
31 public Cliente registrarUsuario(String data, String nombre, String tipo) {
32     Cliente cliente = new Cliente(nombre, tipo, this.obtenerNumeroLibre());
33     if (tipo.equals("fisica")) {
34         cliente.setDNI(data);
35     }
36     else if (tipo.equals("juridica")) {
37         cliente.setCuit(data);
38     }
39     clientes.add(cliente);
40     return cliente;
41 }
```

III.

- I. Magic Number: En el método “calcularMontoTotalLlamadas” de la clase **Empresa** existen números que aparecen de la nada.
- II. Aplico Replace Magic Number With Symbolic Constant.
 - A. Reemplazo el número 0.21 por una constante llamada “IVA”.
 - B. Reemplazo el número 3 por una constante llamada “precioNacional”.
 - C. Reemplazo el número 150 por una constante llamada “precioInternacional”
 - D. Reemplazo el número 50 por una constante llamada “extraInternacional”
- III.

```
6 public class Empresa {
7     private List<Cliente> clientes = new ArrayList<Cliente>();
8     private List<Llamada> llamadas = new ArrayList<Llamada>();
9     private GestorNumerosDisponibles guia = new GestorNumerosDisponibles();
10
11     static double descuentoJur = 0.15;
12     static double descuentoFis = 0;
13     static double IVA = 0.21;
14     static double precioNacional = 3;
15     static double precioInternacional = 150;
16     static double extraInternacional = 50;
17
18     public boolean agregarNumeroTelefono(String str) {}
19
20
21     public String obtenerNumeroLibre() {
22         return guia.obtenerNumeroLibre();
23     }
24
25     public Cliente registrarUsuario(String data, String nombre, String tipo) {}
26
27     public Llamada registrarLlamada(Cliente origen, Cliente destino, String t, int duracion) {}
28
29     public double calcularMontoTotalLlamadas(Cliente cliente) {
30         double c = 0;
31         for (Llamada l : cliente.getLlamadas()) {
32             double auxc = 0;
33             if (l.getTipoDeLlamada() == "nacional") {
34                 // el precio es de 3 pesos por segundo más IVA sin adicional por establecer la llamada
35                 auxc += l.getDuracion() * precioNacional + (l.getDuracion() * precioNacional * IVA);
36             } else if (l.getTipoDeLlamada() == "internacional") {
37                 // el precio es de 150 pesos por segundo más IVA más 50 pesos por establecer la llamada
38                 auxc += l.getDuracion() * precioInternacional + (l.getDuracion() * precioInternacional * IVA) + extraInternacional;
39             }
40
41             if (cliente.getTipo() == "fisica") {
42                 auxc -= auxc * descuentoFis;
43             } else if (cliente.getTipo() == "juridica") {
44                 auxc -= auxc * descuentoJur;
45             }
46             c += auxc;
47         }
48         return c;
49     }
50
51     public int cantidadDeUsuarios() {
52         return clientes.size();
53     }
54 }
```

- I. Switch Statements: En el método “obtenerNumeroLibre” de la clase **GestorNumeroDisponible** hay sentencias switch que pueden ser reemplazadas por polimorfismo.
- II. Aplico Replace Conditional With Polymorphism
 - A. Creo una interfaz **TipoGenerador** con el método obtener Número.
 - B. Creo las clases “Primero”, “Ultimo” y “Aleatorio” que implementen la interfaz **TipoGenerador**
 - C. Muevo la lógica del case “ultimo” del método “obtenerNumeroLibre” de la clase **GestorNumerosDisponibles** al método “obtenerNumero” de la clase **Ultimo** y defino una variable String llamada “linea”.
 - D. Muevo la lógica del case “primero” del método “obtenerNumeroLibre” de la clase **GestorNumerosDisponibles** al método “obtenerNumero” de la clase **Primero** y defino una variable String llamada “linea”.
 - E. Muevo la lógica del case “random” del método “obtenerNumeroLibre” de la clase **GestorNumerosDisponibles** al método “obtenerNumero” de la clase **Aleatorio** y defino una variable String llamada “linea”.
 - F. En la clase **GestorNumeroDisponibles** cambio el tipo de la variable “tipoGenerador” de String a TipoGenerador, actualizo su inicialización a “new Ultimo()” y actualizo el tipo del parámetro recibido en el método “cambiarTipoGenerador” y renombro el parámetro a “tipo” para que sea más claro su propósito.
 - G. En el método “obtenerNumeroLibre” de la clase **GestorNumerosDisponibles** elimino todo su código y hago que retorne “this.tipoGenerador.obtenerNumero” y le paso como parametro la variable “lineas”.

```

8 public class GestorNumerosDisponibles {
9     private SortedSet<String> lineas = new TreeSet<String>();
10    private TipoGenerador tipoGenerador = new Ultimo();
11
12    public SortedSet<String> getLineas() {
13        return lineas;
14    }
15
16    public String obtenerNumeroLibre() {
17        return this.tipoGenerador.obtenerNumero(lineas);
18    }
19
20    public void cambiarTipoGenerador(TipoGenerador tipo) {
21        this.tipoGenerador = tipo;
22    }
23 }

```

III.

- IV. Actualizo los test para que cuando el usuario quiera cambiar el tipo de generador, lo haga instanciando la clase que implemente la interfaz **TipoGenerador**.

```

8 class EmpresaTest {
9     Empresa sistema;
10
11    @BeforeEach
12    public void setUp() {
13        this.sistema = new Empresa();
14        this.sistema.agregarNumeroTelefono("2214444554");
15        this.sistema.agregarNumeroTelefono("2214444555");
16        this.sistema.agregarNumeroTelefono("2214444556");
17        this.sistema.agregarNumeroTelefono("2214444557");
18        this.sistema.agregarNumeroTelefono("2214444558");
19        this.sistema.agregarNumeroTelefono("2214444559");
20    }
21
22
23    void testcalcularMontoTotalLlamadas() {}
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45    void testAgregarUsuario() {}
46
47
48
49
50
51
52
53
54    @Test
55    void obtenerNumeroLibre() {
56        // por defecto es el ultimo
57        assertEquals("2214444559", this.sistema.obtenerNumeroLibre());
58
59        this.sistema.getGestorNumeros().cambiarTipoGenerador(new Primero());
60        assertEquals("2214444554", this.sistema.obtenerNumeroLibre());
61
62        this.sistema.getGestorNumeros().cambiarTipoGenerador(new Aleatorio());
63        assertNotNull(this.sistema.obtenerNumeroLibre());
64    }
65 }

```

- I. Switch Statements: En la clase **Empresa** desde la línea 59 hasta 65 hay sentencias switch que pueden resolverse con polimorfismo.
- II. Aplico Replace Conditional With Polymorphism.
 - A. Creo una interfaz **TipoLlamada**.
 - B. Creo las clases **Nacional** e **Intencional** que implementen **TipoLlamada**.
 - C. Creo un método en **TipoLlamada** llamado “calcularPrecio”.
 - D. Muevo la lógica de llamada nacional al método “calcularPrecio” de la clase **Nacional**.
 - E. Muevo la lógica de llamada nacional al método “calcularPrecio” de la clase **Internacional**.
 - F. Muevo la constante “precioNacional” de la clase **Empresa** a la clase **Nacional**.
 - G. Muevo la constante “precioInternacional” y “extraInternacional” de la clase **Empresa** a la clase **Internacional**.
 - H. Modifico la variable de instancia “tipoLlamada” de la clase **Llamada** para que pase de String a TipoLlamada, junto con el parámetro del constructor y elimino el método “getTipoLlamada”.
 - I. En el método “registrarLlamada” de la clase **Empresa**, modifico el parámetro “t” para que sea de TipoLlamada y lo renombro como “tipoLlamada” para que sea claro su nombre.
 - J. Creo un método en la clase **Llamada** que se llama “calcularPrecio”, este método retorna “this.tipoLlamada.calcularPrecio” y manda como parámetro “iva” y la variable “duracion”.
 - K. En el método “calcularMontoTotalLlamadas” de la clase **Empresa** elimino los condicionales del cálculo de llamada por el llamado al método “l.calcularPrecio” y le paso la constante “IVA” como parámetro.

```

6 public class Empresa {
7     private List<Cliente> clientes = new ArrayList<Cliente>();
8     private List<Llamada> llamadas = new ArrayList<Llamada>();
9     private GestorNumerosDisponibles guia = new GestorNumerosDisponibles();
10
11     static double descuentoJur = 0.15;
12     static double descuentoFis = 0;
13     static double IVA = 0.21;
14
15     public boolean agregarNumeroTelefono(String str) {}
16
17
18     public String obtenerNumeroLibre() {
19         return guia.obtenerNumeroLibre();
20     }
21
22
23     public Cliente registrarUsuario(String data, String nombre, String tipo) {}
24
25
26     public Llamada registrarLlamada(Cliente origen, Cliente destino, TipoLlamada tipoLlamada, int duracion) {
27         Llamada llamada = new Llamada(tipoLlamada, origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
28         llamadas.add(llamada);
29         origen.agregarLlamada(llamada);
30         return llamada;
31     }
32
33
34     public double calcularMontoTotalLlamadas(Cliente cliente) {
35         double c = 0;
36         for (Llamada l : cliente.getLlamadas()) {
37             double auxc = l.calcularPecio(IVA);
38
39             if (cliente.getTipo() == "fisica") {
40                 auxc -= auxc*descuentoFis;
41             } else if (cliente.getTipo() == "juridica") {
42                 auxc -= auxc*descuentoJur;
43             }
44             c += auxc;
45         }
46         return c;
47     }
48
49
50     public int cantidadDeUsuarios() {
51         return clientes.size();
52     }
53
54
55     public boolean existeUsuario(Cliente persona) {
56         return clientes.contains(persona);
57     }
58
59
60     public GestorNumerosDisponibles getGestorNumeros() {
61         return this.guia;
62     }
63 }

```

III.

IV. Modifico el test “testCalcularMontoTotalLlamadas” para que el método “registrarLlamada” reciba como parámetro la clase de la llamada correspondiente en lugar de un string.

```

22 @Test
23 void testcalcularMontoTotalLlamadas() {
24     Cliente emisorPersonaFisca = sistema.registrarUsuario("11555666", "Brendan Eich", "fisica");
25     Cliente remitentePersonaFisca = sistema.registrarUsuario("00000001", "Doug Lea", "fisica");
26     Cliente emisorPersonaJuridica = sistema.registrarUsuario("17555222", "Nvidia Corp", "juridica");
27     Cliente remitentePersonaJuridica = sistema.registrarUsuario("25765432", "Sun Microsystems", "juridica");
28
29     this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaFisca, new Nacional(), 10);
30     this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaFisca, new Internacional(), 8);
31     this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaJuridica, new Nacional(), 5);
32     this.sistema.registrarLlamada(emisorPersonaJuridica, remitentePersonaJuridica, new Internacional(), 7);
33     this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaFisca, new Nacional(), 15);
34     this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaFisca, new Internacional(), 45);
35     this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaJuridica, new Nacional(), 13);
36     this.sistema.registrarLlamada(emisorPersonaFisca, remitentePersonaJuridica, new Internacional(), 17);
37
38     assertEquals(11454.64, this.sistema.calcularMontoTotalLlamadas(emisorPersonaFisca), 0.01);
39     assertEquals(2445.40, this.sistema.calcularMontoTotalLlamadas(emisorPersonaJuridica), 0.01);
40     assertEquals(0, this.sistema.calcularMontoTotalLlamadas(remitentePersonaFisca));
41     assertEquals(0, this.sistema.calcularMontoTotalLlamadas(remitentePersonaJuridica));
42 }

```

- I. Switch Statements: En las líneas 56 a 60 de la clase **Empresa** hay sentencias switch que pueden ser reemplazadas por polimorfismo.
- II. Aplico Replace Conditional With Polymorphism.
 - A. Creo una interfaz llamada **TipoCliente**.
 - B. Creo las clases **Fisico** y **Juridico** que implementen la interfaz **TipoCliente**.
 - C. Creo el método “aplicarDescuento” en la interfaz **TipoCliente**.
 - D. Muevo la constante “descuentoFis” de la clase **Empresa** a la clase **Fisico**.
 - E. Muevo la constante “descuentoJur” de la clase **Empresa** a la clase **Juridico**.
 - F. Cambio el tipo de la variable “tipo” de String a TipoCliente de la clase **Cliente**. Elimino el método “getTipo”, actualizo constructor y método “setTipo”.
 - G. Creo un método llamado “calcularTotalLlamadas” en la clase **Cliente**, que defina una variable “total” en donde almacene el total de sus llamadas y después retorne el total de sus llamadas con el descuento correspondiente.
 - H. En la clase **Empresa** defino los métodos “registrarUsuarioFisico” y “registrarUsuarioJuridico” para poder crear a los clientes según su tipo, y elimino el método “registrarUsuario”.

- I. Actualizo el método “calcularMontoTotalLlamadas” para que llame al método “calcularMontoTotalLlamadas” del cliente pasándole el iva como parámetro.

```
6 public class Empresa {
7     private List<Cliente> clientes = new ArrayList<Cliente>();
8     private List<Llamada> llamadas = new ArrayList<Llamada>();
9     private GestorNumerosDisponibles guia = new GestorNumerosDisponibles();
10
11     static double IVA = 0.21;
12
13     public boolean agregarNumeroTelefono(String str) {}
14
15
16     public String obtenerNumeroLibre() {
17         return guia.obtenerNumeroLibre();
18     }
19
20     public Cliente RegistrarUsuarioFisico(String data, String nombre, TipoCliente tipo)
21     {
22         Cliente cliente = new Cliente(nombre, tipo, this.obtenerNumeroLibre());
23         cliente.setDNI(data);
24         return cliente;
25     }
26
27     public Cliente RegistrarUsuarioJuridico(String data, String nombre, TipoCliente tipo)
28     {
29         Cliente cliente = new Cliente(nombre, tipo, this.obtenerNumeroLibre());
30         cliente.setCuit(data);
31         return cliente;
32     }
33
34     public Llamada registrarllamada(Cliente origen, Cliente destino, TipoLlamada tipoLlamada, int duracion) {
35         Llamada llamada = new Llamada(tipoLlamada, origen.getNumeroTelefono(), destino.getNumeroTelefono(), duracion);
36         llamadas.add(llamada);
37         origen.agregarLlamada(llamada);
38         return llamada;
39     }
40
41     public double calcularMontoTotalLlamadas(Cliente cliente) {
42         return cliente.calcularTotalLlamadas(IVA);
43     }
44
45     public int cantidadDeUsuarios() {
46         return clientes.size();
47     }
48
49     public boolean existeUsuario(Cliente persona) {
50         return clientes.contains(persona);
51     }
52
53     public GestorNumerosDisponibles getGestorNumeros() {
54         return this.guia;
55     }
56 }
57
```

III.

En la clase Cliente

```

14 public Cliente(String nombre, TipoCliente tipo, String numeroTelefono)
15 {
16     this.nombre = nombre;
17     this.tipo = tipo;
18     this.numeroTelefono = numeroTelefono;
19 }
20
21 public void setTipo(TipoCliente tipo) {
22     this.tipo = tipo;
23 }
24 public String getNombre() {
25     return nombre;
26 }
27 public void setNombre(String nombre) {
28     this.nombre = nombre;
29 }
30 public String getNumeroTelefono() {
31     return numeroTelefono;
32 }
33 public void setNumeroTelefono(String numeroTelefono) {
34     this.numeroTelefono = numeroTelefono;
35 }
36 public String getCuit() {
37     return cuit;
38 }
39 public void setCuit(String cuit) {
40     this.cuit = cuit;
41 }
42 public String getDNI() {
43     return dni;
44 }
45 public void setDNI(String dni) {
46     this.dni = dni;
47 }
48
49 public List<Llamada> getLlamadas()
50 {
51     return this.llamadas;
52 }
53
54 public void agregarLlamada(Llamada llamada)
55 {
56     this.llamadas.add(llamada);
57 }
58
59 public double calcularTotalLlamadas(double iva)
60 {
61     double total = this.llamadas.stream().mapToDouble(l -> l.calcularPecio(iva)).sum();
62     return this.tipo.aplicarDescuento(total);
63 }
64 }

```

Ejercicio 11

Malos olores y refactorings aplicados:

- Código repetido: Lo solucione aplicando el patrón null object para no preguntar siempre por null, de esta forma elimino código repetido.

Ejercicio 12

Bad smell	Linea/s	Justificación	Refactoring
Nombre no comunicativo	2 y 3	El nombre no dice nada sobre lo que representa, viéndose obligado a agregar comentarios para explicar lo que es	Rename Field
Variable temporal	18	Variable innecesaria, puede resolver el valor sin la necesidad de utilizarla	Inline Temp
Parámetro no utilizado (Unused Parameter / Código muerto)	17	Parámetro innecesario, ya que no es utilizado en el método	Remove Parameter